PROCEDURAL KNOWLEDGE GENERATION WITH SELECTIVE RETRIEVAL FROM PAST OUTPUTS

Anonymous authors
Paper under double-blind review

ABSTRACT

Retrieval-augmented generation (RAG) has become a common approach to ground the output of large language models (LLMs). For topics unseen in training, RAG requires careful search tuning to ensure the model sees the information necessary to answer the question, especially if it is dispersed across many documents. If we instead answer requests one at a time and store the responses, some information for later outputs will likely be densely available in earlier responses. We propose a new augmentation technique called Selective Retrieval from Past Outputs (SRPO) which searches a library of previous responses using model-generated queries, and filters them by having the model decide if each document is relevant. To demonstrate that this approach more easily surfaces useful information, we publish a new dataset called LCStep containing 120 clean, step-by-step procedures extracted from the LangChain Python documentation (created after the OpenAI knowledge cutoff date), along with a model-based evaluation metric that judges the quality of generated procedures. We show that GPT-4 cannot reliably generate these procedures from a candidate goal, even when augmented with retrieved texts from LangChain's conceptual documentation and API reference. We demonstrate that collecting outputs into a library of retrievable procedures can improve later responses, as long as retrieved procedures are highly relevant to the current generation.

1 LCSTEP

LCStep contains three sets of documents: API reference, conceptual documentation, and procedures. See Figure 1 for a diagram of the process of generating the LCStep data.

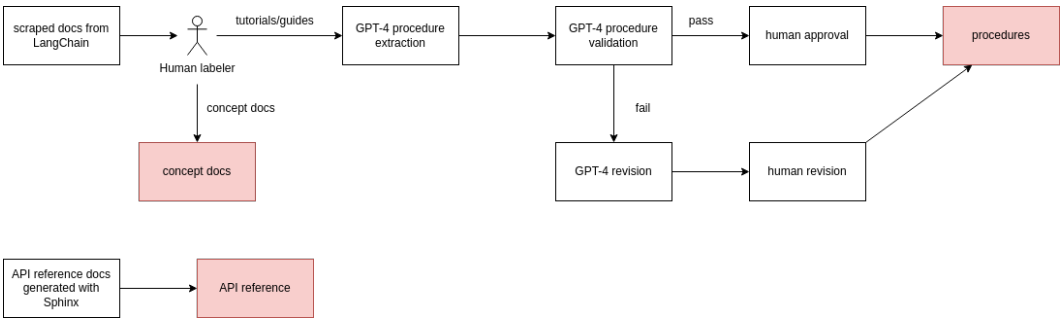

Figure 1: The workflow used to generate the LCStep dataset.

1.1 API REFERENCE

We generate the API reference material from the source files in the LangChain GitHub repository[1] using Sphinx. These files contain descriptions of all APIs in the Python package, including call signatures and argument descriptions. These files do not contain any usage examples or high-level explanation.

[1] https://github.com/langchain-ai/langchain/

1.2 CONCEPTUAL DOCUMENTATION AND PROCEDURES

We collect these documents by scraping the LangChain Python docs[2]. We manually filter out topic pages and stubs, leaving 228 documents. We then manually classified these into 137 documents of conceptual documentation, and 91 documents containing tutorials/guides.

For the 91 tutorials/guides, we prompted GPT-4[3] to extract a list of high-level steps necessary to accomplish the goal. We then prompted GPT-4[4] to rate those extracted procedures using a list of criteria. We found that this caught many mistakes where GPT-4 did not follow all the stated instructions. In those cases, we had the model revise the steps to meet the requirements, and then we manually checked the revised versions.

2 SELECTIVE RETRIEVAL FROM PAST OUTPUTS

TODO

3 EXPERIMENTS

To demonstrate that SRPO can overcome shortcomings of standard retrieval, we pit the two methods against each other on the task of generating the procedures in LCStep. We also test against a few baselines and oracles which we explain below. All methods are used with `GPT-4-0614` from OpenAI as well as a version of LLaMA2 instruction-tuned by Thales. Both methods have access to the API reference and conceptual documentation from the dataset. For comparison, we also test a simple few-shot approach (which should fail due to a lack of knowledge of LangChain in the models' training data).

3.1 BASELINES

The **few-shot** approach will prompt GPT-4 with a basic instruction and a few examples

3.2 EVALUATION

REFERENCES

A LCSTEP PROMPTS

A.1 INSTRUCTIONS FOR FORMATTING PROCEDURES

You are helping format procedures. I will give you raw procedure text, describing how to complete a procedure, with code and examples. Your task is to simplify and generalize this procedure to achieve the desired format. The desired format is: a title, description, a minimal set of simple necessary instructions for general application where every instruction is describing one action and the API references to use in that action (with the module the API reference belongs to), and side notes if needed. Remove all unnecessary details, code, examples and specific cases. Keep only the strictly necessary steps to accomplish the procedure.The procedure should start with a title and a brief description containing any important information that doesn't fit into the instructions. Do not mention importing the required modules as a separate step, as we will include the required imports from the API references made in the instructions. If there are any checks, or case-specific instructions, they should be specified as a side note after the instructions as we want the procedure instructions to be for general use. If an API reference has parameters, make sure to mention these parameters in the instruction. If the raw text is actually describing multiple different procedures, you may then have the formatted procedure split into multiple different sets of instructions for each different procedure, but this should only be done if judged necessary.

[2] https://python.langchain.com/

[3] See Appendix A.1 for the full prompt.

[4] See Appendix A.2 for the full prompt.

A.2 INSTRUCTIONS FOR VALIDATING PROCEDURES

TODO